

CS224N Lecture 8:

Self-Attention and Transformers (Part 1)

Stanford University

Spring 2023

Abstract

This lecture introduces **self-attention** and the **Transformer architecture**, which form the foundation of modern natural language processing. We explore the limitations of recurrent neural networks (RNNs), particularly their linear interaction distance and non-parallelizability. Self-attention addresses these issues by allowing words to directly interact regardless of distance. We cover the key-query-value mechanism, position encodings, feed-forward networks, and masking—the essential components needed to build a complete self-attention architecture.

Contents

1	Motivation: From RNNs to Self-Attention	2
1.1	The RNN/LSTM Paradigm (2014–2017)	2
1.2	Why Replace RNNs?	2
2	Problems with Recurrent Neural Networks	2
2.1	Problem 1: Linear Interaction Distance	2
2.1.1	Why This Matters	2
2.2	Problem 2: Lack of Parallelizability	3
2.2.1	Computational Dependency Graph	3
2.3	Self-Attention: The Solution	3
3	Attention as Fuzzy Lookup	4
3.1	Traditional Key-Value Store	4
3.2	Attention: Soft Key-Value Store	4
3.3	Example: Self-Attention in a Sentence	5
4	Self-Attention Mechanism	5
4.1	Notation and Setup	5
4.2	Key-Query-Value Transformations	5
4.3	Computing Attention	6
4.3.1	Mathematical Formulation	6
4.4	Visual Example: Computing Attention for One Word	7
4.5	Why Separate W^Q and W^K ?	7
5	Building Blocks for Self-Attention	7
5.1	Problem 1: No Sequence Order	7
5.1.1	Solution: Position Encodings	8
5.1.2	Option 1: Sinusoidal Position Encodings	8
5.1.3	Option 2: Learned Position Embeddings	9

5.2	Problem 2: No Non-linearities	9
5.2.1	Solution: Feed-Forward Networks	9
5.3	Problem 3: Masking for Decoders	10
5.3.1	Solution: Masked Self-Attention	10
6	The Complete Self-Attention Block	11
6.1	Minimal Architecture	11
6.2	Key Properties	11
6.3	Computational Complexity	12
7	Summary	12
7.1	Looking Ahead	12
8	Further Reading	13
8.1	Foundational Papers	13
8.2	Advanced Topics	13

1 Motivation: From RNNs to Self-Attention

1.1 The RNN/LSTM Paradigm (2014–2017)

Before 2017, the standard approach to NLP tasks involved:

Pre-Transformer NLP Recipe

1. **Bi-directional LSTM encoder** for input sequences
 - Captures both forward and backward context
 - Used for sentence/document encoding
2. **Uni-directional LSTM decoder** for output generation
 - Generates output token-by-token
 - Used for translation, text generation
3. **Attention mechanism** for flexible memory access
 - Allows decoder to "look back" at encoder
 - Solves bottleneck problem in seq2seq

This approach worked exceptionally well! It was state-of-the-art for machine translation, question answering, and many other tasks.

1.2 Why Replace RNNs?

Despite their success, RNNs have fundamental limitations that motivated the development of self-attention and Transformers.

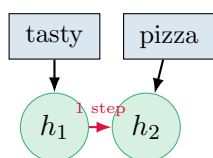
2 Problems with Recurrent Neural Networks

2.1 Problem 1: Linear Interaction Distance

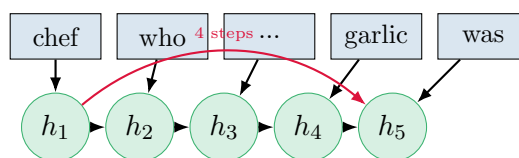
Problem 2.1 (Linear Interaction Distance). In an RNN, the number of sequential steps between two positions determines how easily they can interact. Words that are far apart in a sequence require many RNN steps to exchange information.

2.1.1 Why This Matters

Short dependency:



Long dependency:



"The chef who ... loves garlic was"
chef and **was** should interact
but are separated by many steps

Consequences

- **Gradient flow:** Even with LSTMs, gradients must backpropagate through many steps
- **Learning difficulty:** Hard to learn long-distance dependencies
- **Not linguistically motivated:** Linear distance $\neq$ syntactic/semantic distance

Example 2.1 (Linear vs. Syntactic Distance). Consider: “The chef who went to the store and bought ingredients was happy.”

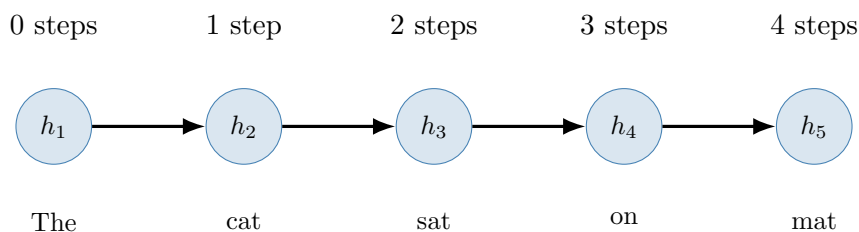
- **Linear distance** from “chef” to “was”: 10 words
- **Syntactic distance:** “chef” is the subject of “was” (direct dependency)

RNNs must traverse the entire linear path, but these words should interact directly!

2.2 Problem 2: Lack of Parallelizability

Problem 2.2 (Sequential Computation). RNNs have $O(n)$ **sequential** operations, where n is the sequence length. Computing h_t requires computing $h_{t-1}, h_{t-2}, \dots, h_1$ first.

2.2.1 Computational Dependency Graph



To compute h_5 , must first compute:
 h_1 , then h_2 , then h_3 , then h_4
Cannot parallelize across sequence!

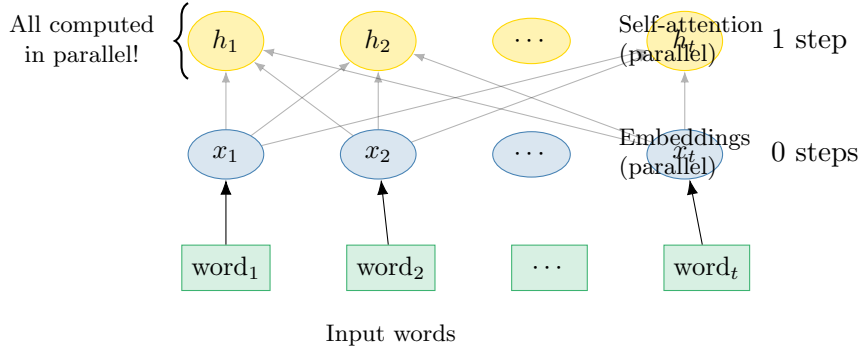
GPU Parallelization Problem

GPUs excel at parallel operations, but RNNs have inherent sequential dependencies:

- Cannot compute all h_t simultaneously
- Must wait for previous time steps
- Limits training speed, especially for long sequences
- Wastes GPU computational capacity

2.3 Self-Attention: The Solution

Self-attention addresses both problems:



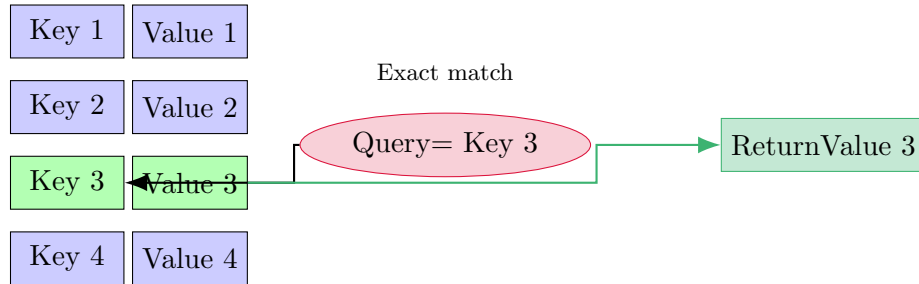
Self-Attention Advantages

1. **Constant path length:** Any word can directly attend to any other word (1 step)
2. **Parallelizable:** All attention operations can be computed simultaneously
3. **Flexible:** Learn which words to attend to, regardless of distance

3 Attention as Fuzzy Lookup

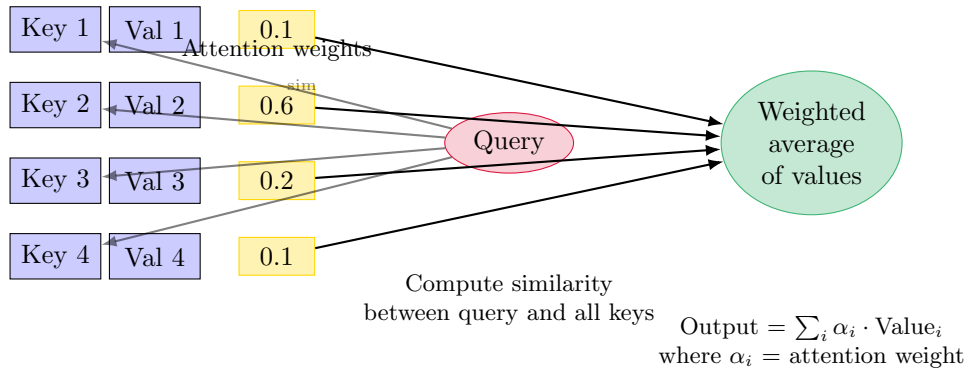
3.1 Traditional Key-Value Store

Definition 3.1 (Key-Value Store). A data structure that maps keys to values. Given a query that exactly matches a key, return the corresponding value.



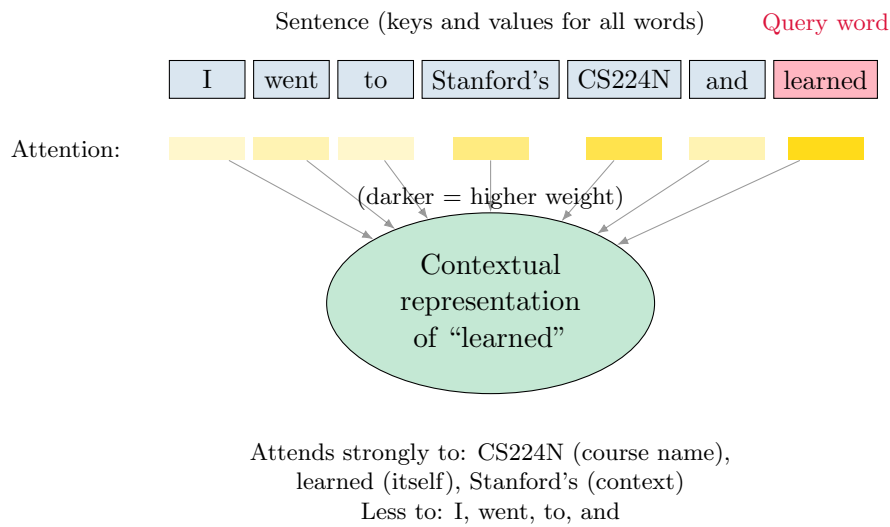
3.2 Attention: Soft Key-Value Store

Definition 3.2 (Attention as Soft Lookup). Given a query, compute similarity with **all** keys (not exact match). Return weighted average of values, weighted by query-key similarities.



Remark 3.1. The key difference: instead of exact matching, attention computes a **soft** distribution over all keys and combines their values.

3.3 Example: Self-Attention in a Sentence



4 Self-Attention Mechanism

4.1 Notation and Setup

Definition 4.1 (Input Sequence). Given a sequence of words $w_1, w_2, \dots, w_n$ from vocabulary V , we:

1. Embed each word: $x_i = Ew_i$ where $E \in \mathbb{R}^{d \times |V|}$
2. Get word embeddings: $x_1, x_2, \dots, x_n$ where $x_i \in \mathbb{R}^d$

4.2 Key-Query-Value Transformations

Three Learned Transformations

For each word embedding $x_i \in \mathbb{R}^d$, compute:

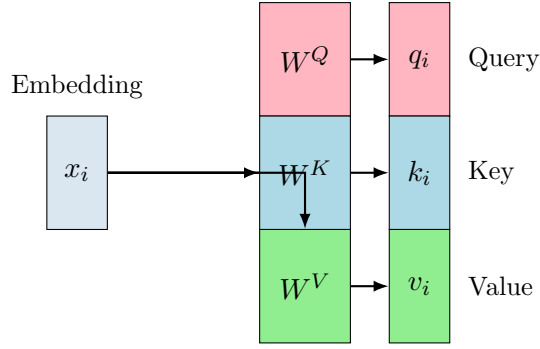
$$\text{Query: } q_i = W^Q x_i \quad W^Q \in \mathbb{R}^{d \times d} \quad (1)$$

$$\text{Key: } k_i = W^K x_i \quad W^K \in \mathbb{R}^{d \times d} \quad (2)$$

$$\text{Value: } v_i = W^V x_i \quad W^V \in \mathbb{R}^{d \times d} \quad (3)$$

Intuition:

- **Query:** “What am I looking for?”
- **Key:** “What do I contain?”
- **Value:** “What information do I actually provide?”



Same embedding x_i transformed
three different ways

4.3 Computing Attention

Algorithm 1 Self-Attention Computation

- 1: **Input:** Embeddings $x_1, \dots, x_n$
 - 2: **Step 1:** Compute queries, keys, values for all words:
 - 3: $q_i = W^Q x_i, \quad k_i = W^K x_i, \quad v_i = W^V x_i$ for $i = 1, \dots, n$
 - 4: **Step 2:** Compute attention scores (all pairs):
 - 5: $e_{ij} = q_i^T k_j$ for all i, j
 - 6: **Step 3:** Normalize with softmax:
 - 7: $\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'=1}^n \exp(e_{ij'})}$
 - 8: **Step 4:** Compute output as weighted sum:
 - 9: $h_i = \sum_{j=1}^n \alpha_{ij} v_j$
 - 10: **Output:** Contextualized representations $h_1, \dots, h_n$
-

4.3.1 Mathematical Formulation

Self-Attention Equations

For word at position i :

1. Attention scores:

$$e_{ij} = q_i^T k_j = (W^Q x_i)^T (W^K x_j) \quad (4)$$

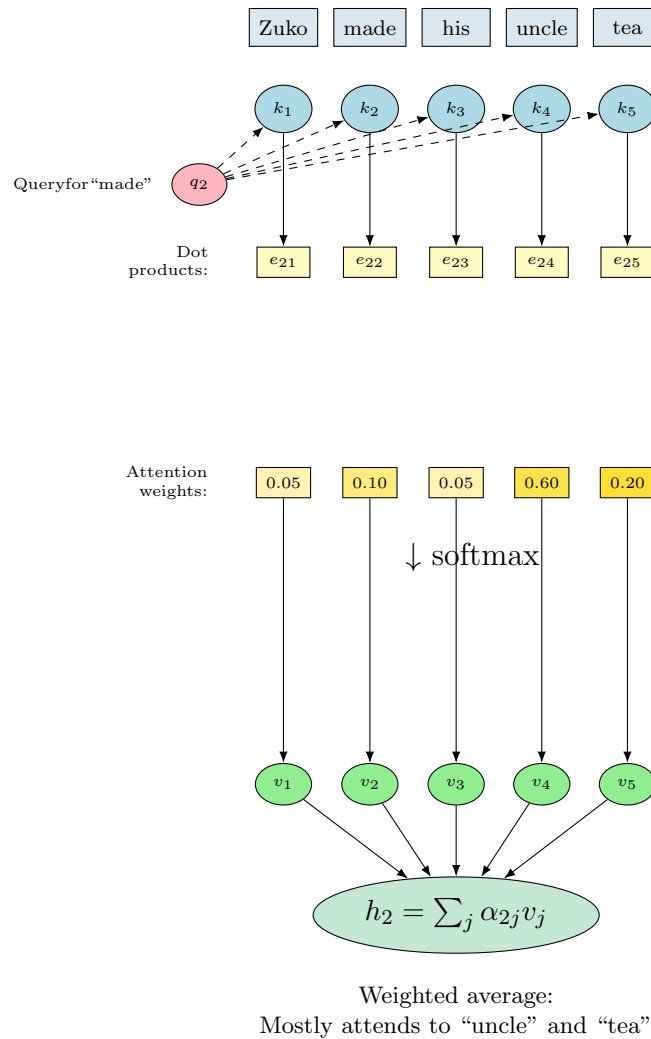
2. Attention weights (softmax):

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'=1}^n \exp(e_{ij'})} \quad (5)$$

3. Output (weighted sum of values):

$$h_i = \sum_{j=1}^n \alpha_{ij} v_j = \sum_{j=1}^n \alpha_{ij} (W^V x_j) \quad (6)$$

4.4 Visual Example: Computing Attention for One Word



4.5 Why Separate W^Q and W^K ?

Remark 4.1. A common question: Why not just compute $e_{ij} = x_i^T W x_j$ with a single matrix?

Answer: Using W^Q and W^K gives a **low-rank approximation**:

$$e_{ij} = q_i^T k_j = x_i^T (W^Q)^T W^K x_j \quad (7)$$

This is equivalent to $x_i^T W x_j$ where $W = (W^Q)^T W^K$, but W has rank at most d .

Benefits:

- Fewer parameters: $2d^2$ vs. d^2 (though both are $O(d^2)$)
- Computational efficiency
- Conceptually cleaner separation of roles

5 Building Blocks for Self-Attention

5.1 Problem 1: No Sequence Order

Problem 5.1 (Permutation Invariance). Self-attention as presented is **permutation-invariant**: the same set of words in different orders produces the same representations.

Example 5.1 (Order Matters!). • "Zuko made his uncle tea" $\neq$ "His uncle made Zuko tea"

- “Dog bites man” $\neq$ “Man bites dog”

But self-attention treats these identically because it only depends on the *set* of embeddings, not their order.

5.1.1 Solution: Position Encodings

Definition 5.1 (Position Encoding). Represent each position $i \in \{1, 2, \dots, n\}$ as a vector $p_i \in \mathbb{R}^d$. Add to word embeddings:

$$\tilde{x}_i = x_i + p_i \quad (8)$$

Now the input to self-attention encodes both word identity (x_i) and position (p_i).

5.1.2 Option 1: Sinusoidal Position Encodings

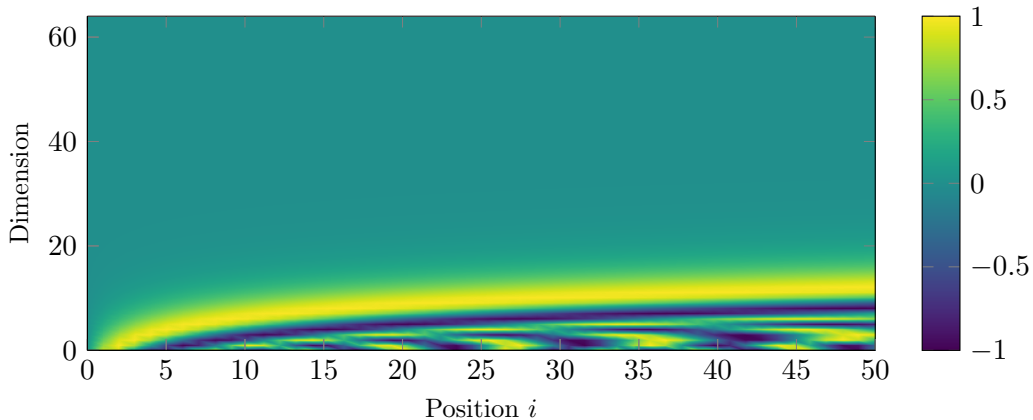
Sinusoidal Positional Encoding (Vaswani et al.)

For position i and dimension j :

$$p_i^{(2j)} = \sin\left(\frac{i}{10000^{2j/d}}\right) \quad (9)$$

$$p_i^{(2j+1)} = \cos\left(\frac{i}{10000^{2j/d}}\right) \quad (10)$$

Intuition: Different frequencies for different dimensions allow representing position.



Sinusoidal position encodings visualization
(Different dimensions have different frequencies)

Pros:

- No learned parameters
- Can extrapolate to longer sequences (theoretically)

Cons:

- Complex, not intuitive
- Doesn't work well for extrapolation in practice

5.1.3 Option 2: Learned Position Embeddings

Learned Positional Embeddings

$$P \in \mathbb{R}^{n_{\max} \times d} \quad (11)$$

where P is a learned parameter matrix. For position i :

$$p_i = P[i, :] \quad (\text{the } i\text{-th row of } P) \quad (12)$$

Learned like any other neural network parameter via backpropagation.

Pros:

- Simple, intuitive
- Learns what's useful for the task
- Most commonly used in practice

Cons:

- Cannot handle sequences longer than $n_{\max}$
- Must choose maximum length ahead of time

Remark 5.1. In practice, $n_{\max}$ is often a bottleneck. Modern large language models typically have:

- GPT-2: $n_{\max} = 1024$
- GPT-3: $n_{\max} = 2048$
- GPT-4: $n_{\max} \approx 8000$ (for some variants)

Attention is $O(n^2)$ in memory, making very long sequences expensive!

5.2 Problem 2: No Non-linearities

Problem 5.2 (Linear Operations Only). Self-attention only computes weighted averages. Without non-linearities, stacking multiple self-attention layers just re-averages the same values.

5.2.1 Solution: Feed-Forward Networks

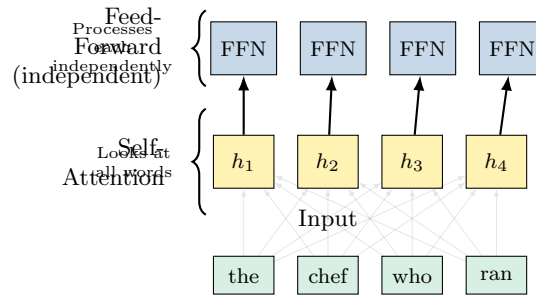
After computing self-attention output h_i , apply a position-wise feed-forward network:

Position-wise Feed-Forward Network

$$\text{FFN}(h_i) = W_2 \cdot \text{ReLU}(W_1 h_i + b_1) + b_2 \quad (13)$$

where:

- $W_1 \in \mathbb{R}^{d_{ff} \times d}$ (typically $d_{ff} = 4d$)
- $W_2 \in \mathbb{R}^{d \times d_{ff}}$
- Applied **independently** to each position



Key Points:

- FFN applied **independently** to each position (fully parallelizable)
- Adds non-linearity and additional expressiveness
- Often contains most of the model's parameters

5.3 Problem 3: Masking for Decoders

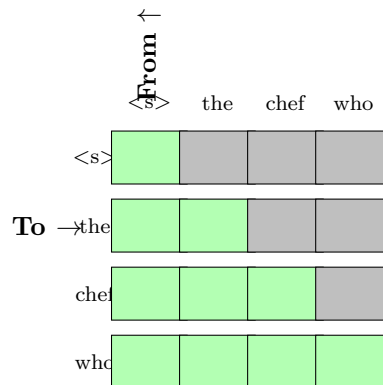
Problem 5.3 (Causality in Generation). For autoregressive tasks (language modeling, translation), we cannot look at future words when predicting the current word.

5.3.1 Solution: Masked Self-Attention

Definition 5.2 (Masked Attention). When computing attention scores e_{ij} , set future positions to $-\infty$ before softmax:

$$e_{ij} = \begin{cases} q_i^T k_j & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (14)$$

After softmax, $\exp(-\infty) = 0$, so future positions get zero attention weight.



Green (): Can attend (score computed normally)
Gray (): Masked (score set to $-\infty$)

When encoding "chef", can only see: $\langle s \rangle$, the, chef

When to Use Masking

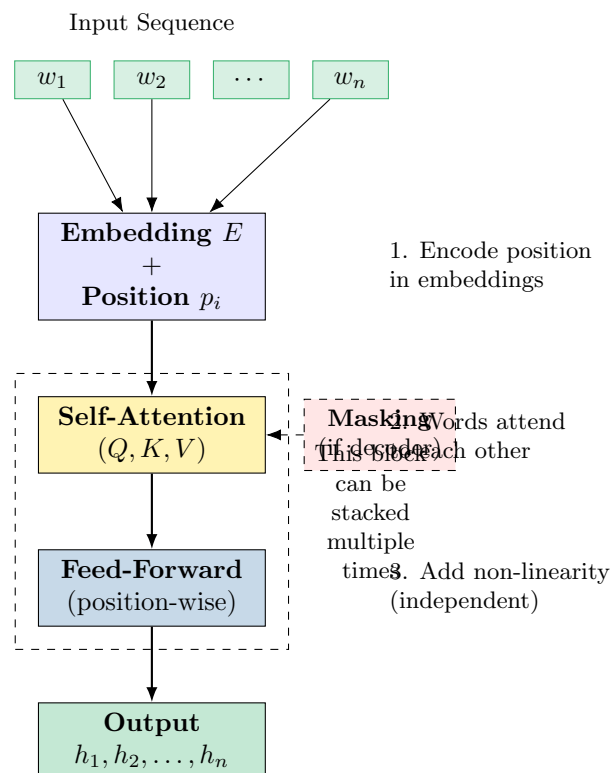
- **Encoders:** Usually **no** masking (bidirectional)
 - Example: Encoding source sentence in translation
 - Want full context from both directions
- **Decoders:** Usually **with** masking (causal)
 - Example: Generating target sentence in translation
 - Must not see future words during training

Remark 5.2. Masking allows us to train on entire sequences in parallel while maintaining the autoregressive property. Without masking, we'd have to train one position at a time!

6 The Complete Self-Attention Block

6.1 Minimal Architecture

We now have all components needed for a complete self-attention building block:



6.2 Key Properties

Self-Attention Block Properties

1. **Position encodings:** Solve sequence order problem
2. **Self-attention:** All-to-all interactions, constant path length
3. **Feed-forward:** Non-linearities and expressiveness
4. **Masking (optional):** Causality for autoregressive generation
5. **Parallelizable:** All positions computed simultaneously
6. **Stackable:** Can repeat this block many times (6, 12, 24+ layers)

6.3 Computational Complexity

Operation	Time	Space	Sequential
RNN	$O(n \cdot d^2)$	$O(n \cdot d)$	$O(n)$
Self-Attention	$O(n^2 \cdot d)$	$O(n^2)$	$O(1)$
Feed-Forward	$O(n \cdot d^2)$	$O(n \cdot d)$	$O(1)$

Table 1: Complexity comparison (n = sequence length, d = embedding dimension)

Key Observations:

- Self-attention is **quadratic** in sequence length (n^2)
- But **constant** sequential operations (fully parallel)
- Trade-off: More memory for better parallelization
- For typical cases ($n < d$), self-attention is competitive with RNN

7 Summary

Key Takeaways

1. Problems with RNNs:

- Linear interaction distance: far words need many steps to interact
- Non-parallelizable: $O(n)$ sequential operations

2. Self-Attention Solution:

- **Attention as soft lookup:** Query matches all keys, returns weighted average of values
- **Key-Query-Value:** Three learned transformations of input embeddings
- **Constant path length:** Any word can directly attend to any other word
- **Fully parallelizable:** All positions computed simultaneously

3. Essential Building Blocks:

- **Position encodings:** Add sequence order information (learned or sinusoidal)
- **Feed-forward networks:** Add non-linearity and expressiveness
- **Masking:** Enable causal attention for autoregressive decoding

4. Complete Architecture:

Input $\xrightarrow{\text{Embed} + \text{Position}}$ $\xrightarrow{\text{Self-Attention}}$ $\xrightarrow{\text{Feed-Forward}}$ Output

Can be stacked to create deep networks (Transformers).

7.1 Looking Ahead

In the next lecture, we'll cover:

- Complete Transformer architecture (encoder-decoder)
- Multi-head attention
- Residual connections and layer normalization

- Training details and tricks
- Applications: BERT, GPT, and beyond

8 Further Reading

8.1 Foundational Papers

1. **Attention Is All You Need:**
Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.
The original Transformer paper.
2. **BERT:**
Devlin, J., et al. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL*.
3. **GPT-2:**
Radford, A., et al. (2019). Language models are unsupervised multitask learners. *OpenAI Blog*.
4. **The Illustrated Transformer:**
<https://jalammar.github.io/illustrated-transformer/>
Excellent visual guide to Transformers.

8.2 Advanced Topics

- **Efficient Transformers:** Methods to reduce $O(n^2)$ complexity
 - Linformer, Reformer, Longformer
 - Sparse attention patterns
- **Position Encodings:**
 - Relative position encodings
 - Rotary Position Embeddings (RoPE)
 - ALiBi (Attention with Linear Biases)
- **Architectural Variants:**
 - Encoder-only (BERT)
 - Decoder-only (GPT)
 - Encoder-decoder (T5)